API SECURITY RISK ANALYSIS

Future Interns Cyber Security Internship

Task 3 — FUTURE_CS_03

Assessment target	OWASP crAPI — local training environment
Target URL	http://127.0.0.1:8888
Assessment type	Authorized API security testing
Report status	Preliminary findings; validation required

Prepared by: Aditya Ashok Jondhale

Project: Future Interns Cyber Security Internship — Task 3

Target: OWASP crAPI (local lab)

Target URL: http://127.0.0.1:8888

Assessment Type: Authorized API security testing

Status: Preliminary findings

1. Executive Summary

This assessment examined selected API behaviors in the intentionally vulnerable OWASP crAPI training environment. Testing covered authentication enforcement, JWT storage, community-post data exposure, pagination parameter handling, and preliminary rate-limit observations.

The assessment identified potential excessive exposure of author metadata and inconsistent handling of pagination parameters. The JWT access token was also observed in browser Local Storage, which can increase token exposure if a same-origin XSS vulnerability exists. No XSS vulnerability was demonstrated.

These findings require validation before final severity assignments. Rate limiting was not conclusively assessed.

2. Scope

The assessment was limited to the local OWASP crAPI instance at `http://127.0.0.1:8888`.

Testing included:

- Browser DevTools request and response inspection
- Authentication header behavior
- JWT storage observation
- Community-post API response inspection
- Pagination parameter tests
- Limited unauthenticated requests

No unrelated or third-party systems were tested.

3. Methodology

1. Started the OWASP crAPI services using Docker Compose.
2. Created and used a test account in the local lab.
3. Inspected API requests and responses through Firefox DevTools.
4. Compared authenticated and unauthenticated requests.
5. Tested selected pagination values.
6. Recorded observations and potential risks for further validation.

4. Findings

Finding 1: Potential Excessive Data Exposure

Endpoint: `GET /community/api/v2/community/posts/recent`

Evidence: The response included community-post content and author metadata, including `nickname`, `email`, `vehicleid`, `profile_pic_url`, and `authorid`.

Potential impact: If email addresses or vehicle identifiers are not intended to be public, their exposure could disclose unnecessary personal or vehicle-related information.

Severity: Tentative Medium; intended visibility and access-control requirements have not been confirmed.

Recommendation: Apply data minimization and return only author fields required by the community feature. Confirm which fields should be public.

Finding 2: JWT Stored in Browser Local Storage

Evidence: The application's browser Local Storage contained an access token identified as a JWT.

Potential impact: JavaScript executing in the same origin may be able to read a token stored in Local Storage. If an XSS vulnerability exists, this could increase the risk of account impersonation.

Severity: Informational observation; no XSS vulnerability was demonstrated.

Recommendation: Apply strong XSS protections, a restrictive Content Security Policy, appropriate token expiration and revocation, and review whether a secure HttpOnly-cookie-based design is suitable.

Finding 3: Pagination Parameter Handling

Endpoint: `GET /community/api/v2/community/posts/recent`

Observed results:

- `limit=abc`: HTTP 200; three posts remained in the response.
- `limit=1000`: HTTP 200; response reported `total: 3`.
- `limit=0`: HTTP 200; response reported `total: 3`.
- `offset=-1`: the response reported that the skip value must be non-negative.
- `offset=9999`: an empty posts array was returned.

Potential impact: Inconsistent handling of invalid pagination values may cause unpredictable API behavior. A security impact has not been demonstrated.

Severity: Low / unconfirmed.

Recommendation: Validate pagination parameters as integers, require positive limits, enforce a reasonable maximum page size, and return consistent validation errors.

5. Authentication Assessment

The community endpoint returned HTTP 200 for an authenticated request. When the `Authorization` header was disabled in the test request, the endpoint returned HTTP 401 Unauthorized.

This supports the conclusion that authentication was enforced in the tested scenario. Object-level authorization was not conclusively assessed.

6. Rate-Limiting Assessment

Five unauthenticated requests returned HTTP 401. The inspected response did not show explicit rate-limit headers.

Conclusion: Rate limiting remains unverified. The results do not establish that rate limiting is absent. Additional controlled authenticated testing is required.

7. Recommendations Summary

1. Minimize author data returned by community endpoints.
2. Review JWT storage, expiration, and revocation.
3. Enforce strict pagination validation and maximum limits.
4. Perform controlled testing to verify rate limiting.
5. Avoid logging access tokens or other secrets.
6. Retest relevant behavior after remediation.

8. Limitations

This assessment used a local intentionally vulnerable training application and a limited set of test cases. The findings are preliminary. No XSS exploit, successful unauthorized access to another user's data, or absence of rate limiting was demonstrated.

9. Conclusion

The assessment documented several API behaviors that warrant review, particularly the author metadata returned by the community endpoint and pagination parameter handling. Further validation is required before assigning final severity or treating the observations as confirmed vulnerabilities.

The testing was restricted to the local OWASP crAPI environment.